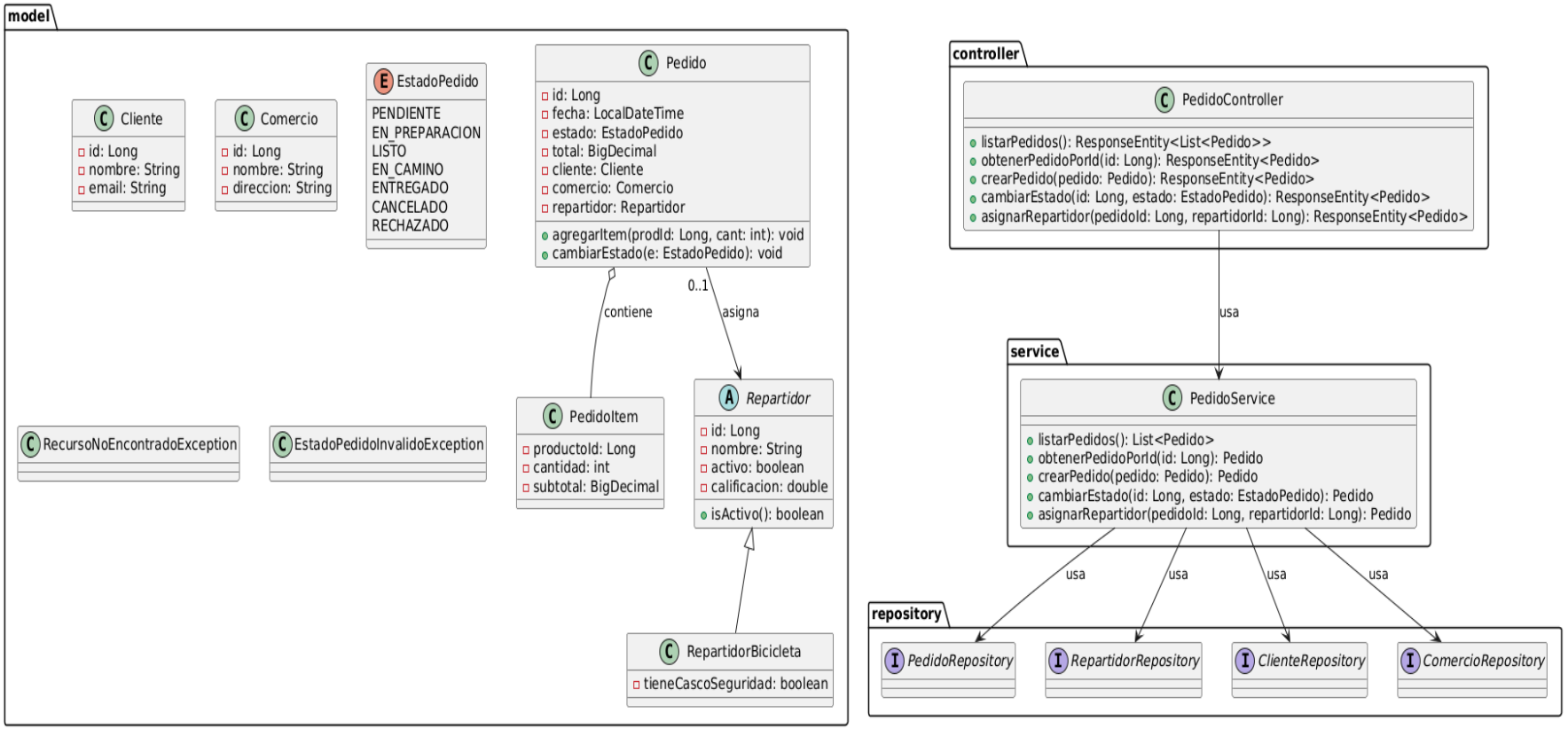
Documentación Técnica:

1.1 Diagrama de Clases UML:

Código:

---------- Lo voy a colocar ----------



Una explicación breve de cada bloque:

controller: una capa que expone la API REST; recibe peticiones HTTP y devuelve respuestas HTTP. También validar entrada básica (formatos, campos obligatorios).

service: una capa que contiene la **lógica de negocio** y las reglas del dominio. Uno de sus roles son las validaciones complejas (transiciones de estado, reglas de asignación de repartidor).

repository: estas son interfaces que abstraen el acceso a la base de datos; normalmente Spring Data JPA.

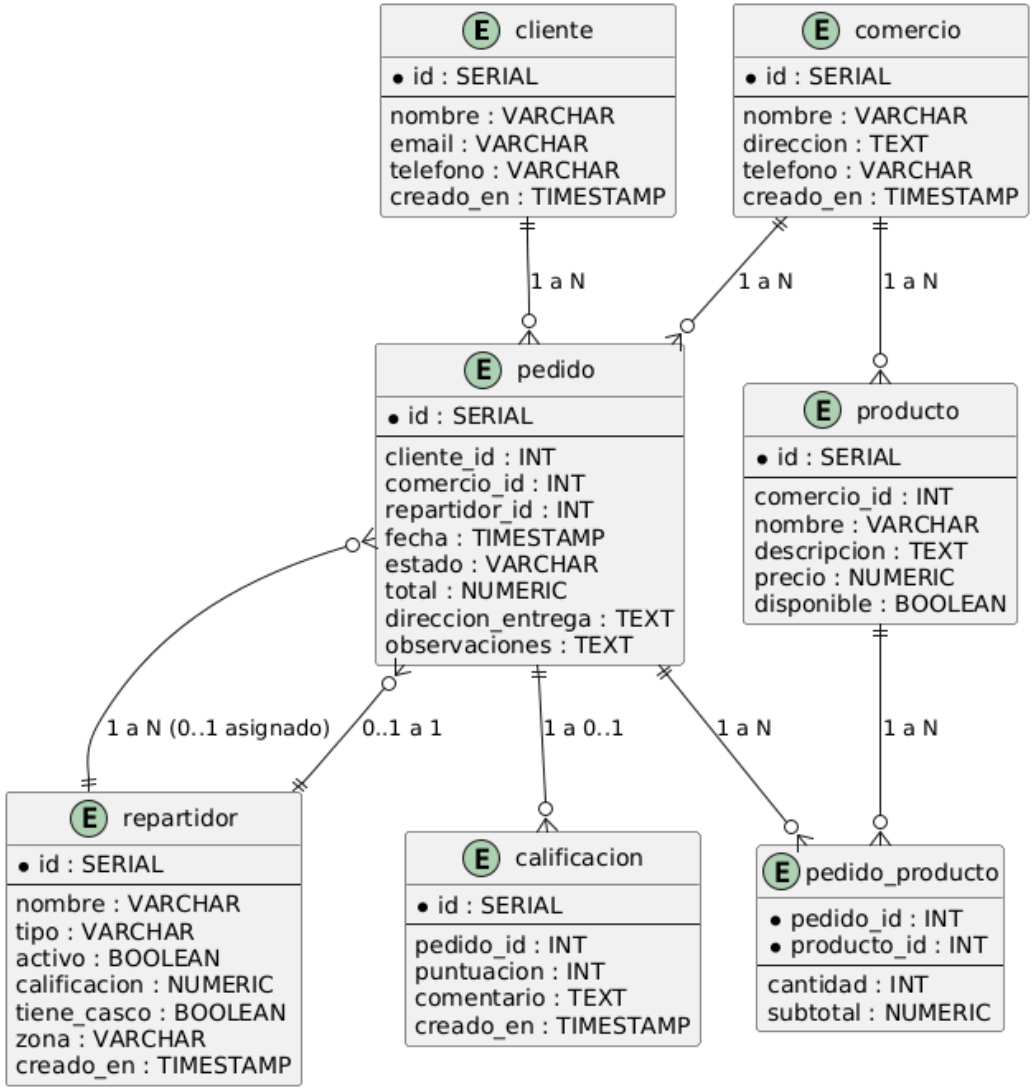
model: son clases del dominio que representan la estructura de datos: como entidades JPA, DTOs y enums. En sus principales responsabilidades el DTOs y Enums para los estados permitidos.

excepciones: son clases que representan errores de negocio o recursos no encontrados. Un ejemplo de este seria: `RecursoNoEncontradoException` → lanzar en service cuando **`pedido`**, **`cliente`** o **`repartidor`** no existen; mapear a HTTP 404.

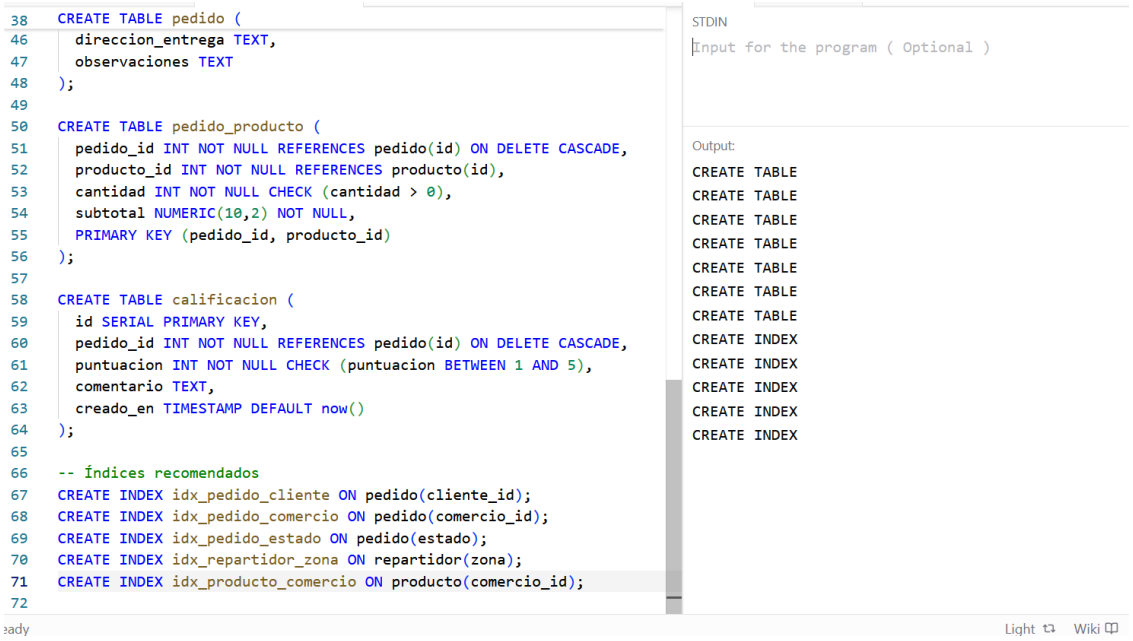
Modelo ER:

En nuestro proyecto y los diagrmas visuales podemos encontrar las Cardinalidades y reglas de negocio relevantes:

- **Cliente 1 — N Pedido**: un cliente puede tener muchos pedidos.
- **Comercio 1 — N Producto**: un comercio publica muchos productos.
- **Pedido 1 — N PedidoProducto**: un pedido contiene varios items; los items no existen fuera del pedido (composición lógica).
- **Producto 1 — N PedidoProducto**: un producto puede aparecer en muchos pedidos.
- **Pedido 0..1 — 1 Repartidor**: un pedido puede tener 0 o 1 repartidor asignado; un repartidor puede atender muchos pedidos.
- **Pedido 0..1 — 0..1 Calificacion**: un pedido entregado puede tener una calificación (opcional).
- **Reglas**: `estado` validado por la capa **service**; **`repartidor.tiene_casco`** usado para validaciones específicas; **`repartidor.activo`** requerido para asignación.



Depuraciones del DR y registros que llevamos con el grupo:



```
38  CREATE TABLE pedido (
46    direccion_entrega TEXT,
47    observaciones TEXT
48  );
49
50  CREATE TABLE pedido_producto (
51    pedido_id INT NOT NULL REFERENCES pedido(id) ON DELETE CASCADE,
52    producto_id INT NOT NULL REFERENCES producto(id),
53    cantidad INT NOT NULL CHECK (cantidad > 0),
54    subtotal NUMERIC(10,2) NOT NULL,
55    PRIMARY KEY (pedido_id, producto_id)
56  );
57
58  CREATE TABLE calificacion (
59    id SERIAL PRIMARY KEY,
60    pedido_id INT NOT NULL REFERENCES pedido(id) ON DELETE CASCADE,
61    puntuacion INT NOT NULL CHECK (puntuacion BETWEEN 1 AND 5),
62    comentario TEXT,
63    creado_en TIMESTAMP DEFAULT now()
64  );
65
66  -- Índices recomendados
67  CREATE INDEX idx_pedido_cliente ON pedido(cliente_id);
68  CREATE INDEX idx_pedido_comercio ON pedido(comercio_id);
69  CREATE INDEX idx_pedido_estado ON pedido(estado);
70  CREATE INDEX idx_repartidor_zona ON repartidor(zona);
71  CREATE INDEX idx_producto_comercio ON producto(comercio_id);
72
```

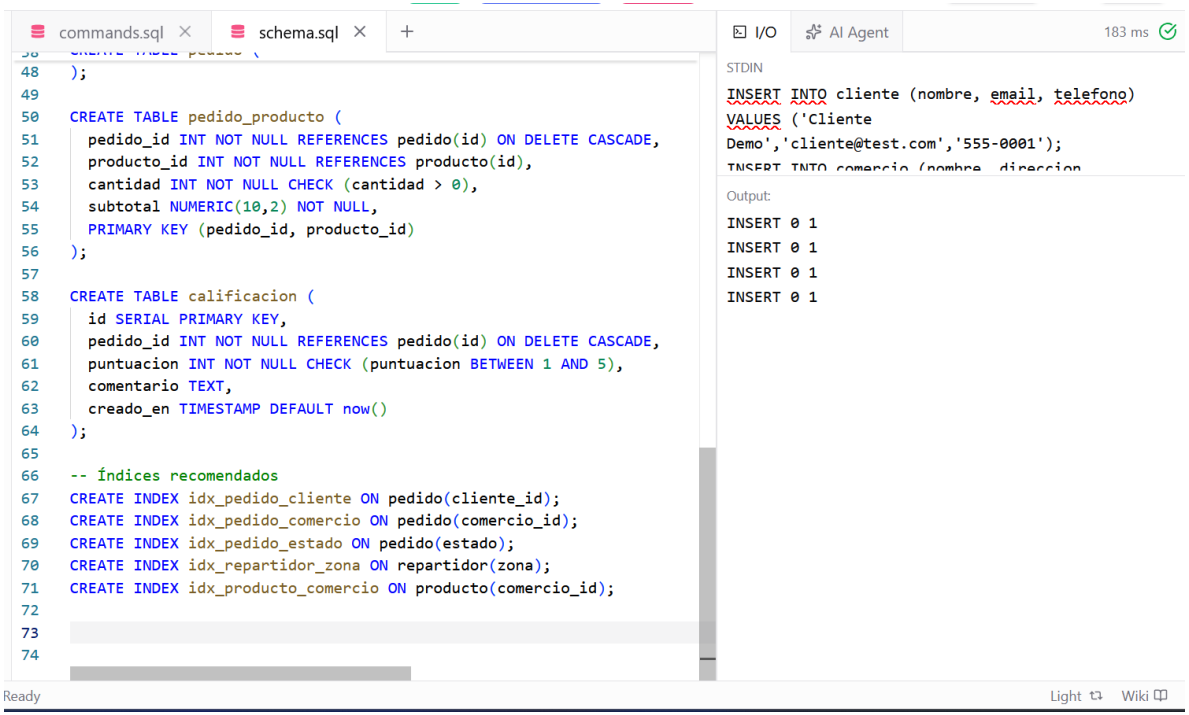
STDIN

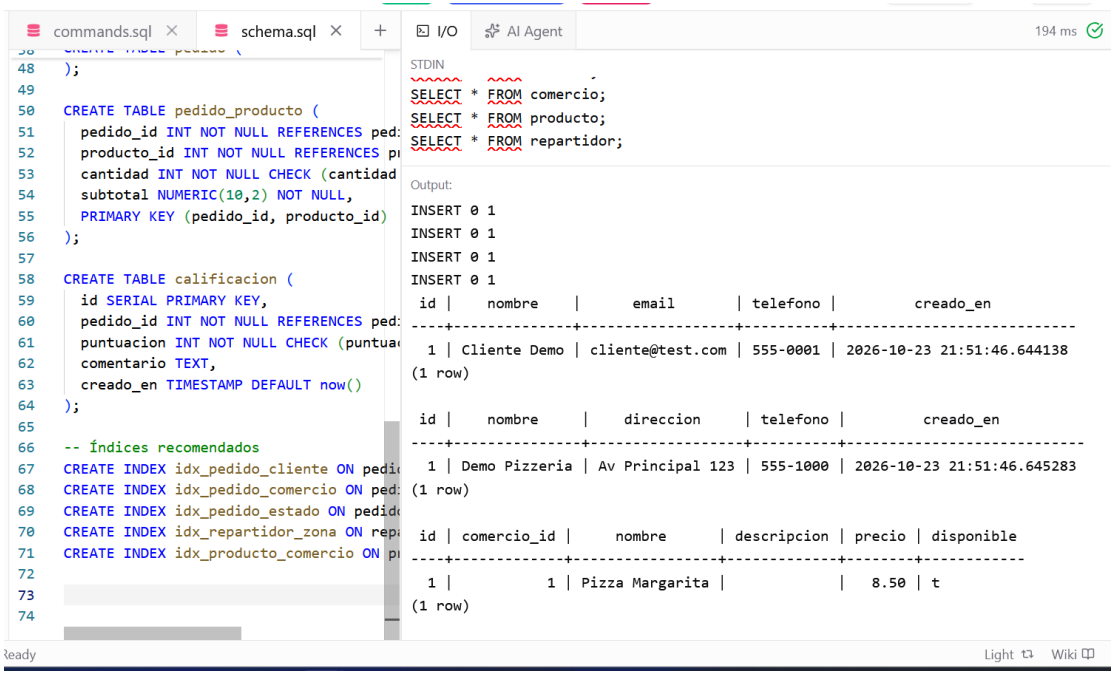
Input for the program (Optional)

Output:

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
```

eady Light Wiki


commands.sql
schema.sql
I/O
AI Agent
183 ms
);
CREATE TABLE pedido_producto (
pedido_id INT NOT NULL REFERENCES pedido(id) ON DELETE CASCADE,
producto_id INT NOT NULL REFERENCES producto(id),
cantidad INT NOT NULL CHECK (cantidad > 0),
subtotal NUMERIC(10,2) NOT NULL,
PRIMARY KEY (pedido_id, producto_id)
);
CREATE TABLE calificacion (
id SERIAL PRIMARY KEY,
pedido_id INT NOT NULL REFERENCES pedido(id) ON DELETE CASCADE,
puntuacion INT NOT NULL CHECK (puntuacion BETWEEN 1 AND 5),
comentario TEXT,
creado_en TIMESTAMP DEFAULT now()
);
-- Índices recomendados
CREATE INDEX idx_pedido_cliente ON pedido(cliente_id);
CREATE INDEX idx_pedido_comercio ON pedido(comercio_id);
CREATE INDEX idx_pedido_estado ON pedido(estado);
CREATE INDEX idx_repartidor_zona ON repartidor(zona);
CREATE INDEX idx_producto_comercio ON producto(comercio_id);
STDIN
INSERT INTO cliente (nombre, email, telefono)
VALUES ('Cliente
Demo','cliente@test.com','555-0001');
Output:
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
Ready
Light
Wiki


commands.sql
schema.sql
I/O
AI Agent
194 ms
);
CREATE TABLE pedido_producto (
pedido_id INT NOT NULL REFERENCES ped
producto_id INT NOT NULL REFERENCES pr
cantidad INT NOT NULL CHECK (cantidad
subtotal NUMERIC(10,2) NOT NULL,
PRIMARY KEY (pedido_id, producto_id)
);
CREATE TABLE calificacion (
id SERIAL PRIMARY KEY,
pedido_id INT NOT NULL REFERENCES ped
puntuacion INT NOT NULL CHECK (puntua
comentario TEXT,
creado_en TIMESTAMP DEFAULT now()
);
-- Índices recomendados
CREATE INDEX idx_pedido_cliente ON pedi
CREATE INDEX idx_pedido_comercio ON ped
CREATE INDEX idx_pedido_estado ON pedid
CREATE INDEX idx_repartidor_zona ON rep
CREATE INDEX idx_producto_comercio ON pr
STDIN
SELECT * FROM comercio;
SELECT * FROM producto;
SELECT * FROM repartidor;
Output:
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
id | nombre | email | telefono | creado_en
1 | Cliente Demo | cliente@test.com | 555-0001 | 2026-10-23 21:51:46.644138
(1 row)
id | nombre | direccion | telefono | creado_en
1 | Demo Pizzeria | Av Principal 123 | 555-1000 | 2026-10-23 21:51:46.645283
(1 row)
id | comercio_id | nombre | descripcion | precio | disponible
1 | 1 | Pizza Margarita | | 8.50 | t
(1 row)
Ready
Light
Wiki

Documentación POO Aplicada – SmartDelivery 360 (no me sé el nombre de nuestra APP :V)

Este documento explica cómo se aplican los principios de **Programación Orientada a Objetos (POO)** en el backend de SmartDelivery 360. El objetivo es justificar las decisiones de diseño y mostrar ejemplos concretos extraídos del código del proyecto (clases **`Pedido`**, **`Repartidor`**, **`PedidoService`**, etc.). El enfoque prioriza **modularidad**, **reutilización**, **claridad de responsabilidades** y **facilidad para pruebas**.

Abstracción:

Definición y propósito La abstracción consiste en modelar las entidades del dominio (cliente, comercio, pedido, repartidor, producto) como clases que encapsulan estado y comportamiento relevante. Cada clase representa un concepto real y expone solo lo necesario para interactuar con él.

Aplicación en el proyecto

- Clase `Pedido`: agrupa atributos como `fecha`, `estado`, `total`, `cliente`, `comercio`, `repartidor` y operaciones como `agregarItem()` y `cambiarEstado()`.
- Beneficio: el resto del sistema (controladores, servicios) trabaja con objetos `Pedido` sin conocer detalles internos de cálculo de totales o validaciones de estado.

Ejemplo conceptual:

```
public class Pedido {
  private Long id;
  private LocalDateTime fecha;
  private EstadoPedido estado;
  private BigDecimal total;
  private Cliente cliente;
  private Comercio comercio;
  private Repartidor repartidor;
  // métodos relevantes...
}
```

Encapsulamiento:

¿Qué y por qué?: El **encapsulamiento** protege el estado interno de las clases mediante atributos `private` y acceso controlado por **getters/setters**. Las validaciones se realizan dentro de los setters o métodos de la propia clase para garantizar invariantes.

Ejemplo práctico**: validación de transiciones de estado** la regla de negocio exige que ciertas transiciones sean inválidas (por ejemplo, no retroceder desde **`ENTREGADO`**). Implementar la validación en **`Pedido`** o en el **`PedidoService`** evita duplicación y mantiene la coherencia. Java:

```
public void setEstado(EstadoPedido nuevoEstado) {
   if (!EstadoPedidoTransitionValidator.esValida(this.estado, nuevoEstado)) {
      throw new EstadoPedidoInvalidoException("Transición inválida: " + this.estado + " -> " + nuevoEstado);
   }
   this.estado = nuevoEstado;
}
```

Beneficio:

- Evita que capas externas pongan el objeto en un estado inconsistente.
- Facilita pruebas unitarias sobre las reglas de negocio encapsuladas.

Herencia y clases abstractas:

Uso y motivación La **herencia** se emplea para modelar jerarquías donde existen comportamientos y atributos comunes. Se define una clase abstracta **`Repartidor`** que contiene la lógica compartida y subclases concretas (**`RepartidorMoto`**, **`RepartidorBicicleta`**, **`RepartidorAutomovil`**) que implementan particularidades.

Ejemplo:

```
public abstract class Repartidor {
  private Long id;
  private String nombre;
  private boolean activo;
  private double calificacion;
  public boolean isActivo() { return activo; }
  public abstract double calcularTiempoEstimado(Pedido pedido);
}
public class RepartidorBicicleta extends Repartidor {
  private boolean tieneCascoSeguridad;
  @Override
  public double calcularTiempoEstimado(Pedido pedido) {
    // implementación específica para bicicleta
  }
  public boolean isTieneCascoSeguridad() { return tieneCascoSeguridad; }
}
```

Beneficio:

- Reutilización de código común.
- Extensibilidad: agregar nuevos tipos de repartidor sin modificar consumidores existentes.

Interfaces y polimorfismo:

Interfaces como contratos: Se definen interfaces como **`Rastreable`**, **`Calificable`** o **`Notificable`** para separar responsabilidades y permitir que distintos objetos cumplan los mismos contratos.

Polimorfismo en el motor de asignación

El motor de asignación puede trabajar con referencias a **`Repartidor`** o a una interfaz **`Asignable`** sin conocer la implementación concreta. Esto evita **`if/else`** por tipo y facilita pruebas y sustitución de implementaciones.

Ejemplo de Interfaz en el Java:

```
public interface Rastreable {
  Ubicacion getUbicacionActual();
}
```

Beneficios:

- Código más flexible y desacoplado.
- Facilita inyección de dependencias y mocks en pruebas.

Excepciones de negocio y manejo de errores

Excepciones específicas: Se usan excepciones propias para distinguir errores técnicos de violaciones de reglas de negocio: **`RecursoNoEncontradoException`**, **`EstadoPedidoInvalidoException`**, etc.

Donde se lanzan y como se manejan

- En la **capa service** se lanzan las excepciones cuando se detectan violaciones (cliente inexistente, repartidor inactivo, transición inválida).

Ejemplo de uso en Java: PedidoService

```
if (!repartidor.isActivo()) {
    throw new IllegalStateException("El repartidor no se encuentra activo");
}
```

Beneficio

- Claridad en el flujo de errores.
- Mejora la experiencia del cliente API con códigos y mensajes coherentes.

Colección y estructura de datos:

Elección de colecciones

- **`List<PedidoItem>`** dentro de `Pedido` para mantener orden y permitir duplicados (mismo producto varias veces).
- **`Map<Zona, List<Repartidor>>`** en el motor de asignación para agrupar repartidores por zona y optimizar búsqueda.

Beneficio

- Estructuras adecuadas mejoran rendimiento y claridad del algoritmo de asignación.

Resumen y Conclusión de nuestra Plataforma:

La aplicación de POO en nuestra plataforma busca mantener el dominio expresivo, **proteger invariantes y facilitar la evolución** del sistema. Las decisiones tomadas, abstracción de entidades, encapsulamiento de reglas, herencia para repartidores, uso de interfaces y excepciones de negocio, permiten un código modular, testeable y alineado con los requisitos del proyecto. Para completar la documentación técnica, se recomienda incluir diagramas UML y ejemplos de pruebas unitarias que verifiquen las reglas descritas (transiciones de estado, validaciones de asignación, manejo de excepciones).